

FinGenie AI

Intelligent Digital Banking & Wealth Management Platform

Project Documentation

Java Full Stack Case Study
Frontend: ReactJS | Backend: Spring Boot

Field	Details
Project Name	FinGenie AI
Project Type	Full Stack Digital Banking and Wealth Management Application
Frontend	ReactJS, JSX, Axios, React Router
Backend	Spring Boot REST APIs, Spring Security, JWT, JPA Repository Pattern
Major Roles	Customer/User and Admin

1. Abstract

FinGenie AI is a full-stack intelligent digital banking and wealth management platform developed using ReactJS for the frontend and Spring Boot for the backend. The application provides secure user and admin authentication, user-specific banking services, transaction processing, loan management, investment support, fraud risk analysis, and chatbot assistance. The project simulates a modern banking ecosystem where customers manage financial activities and administrators monitor platform-wide data through separate dashboards.

The backend follows a layered architecture consisting of controller, DTO, service interface, service implementation, repository, entity, security, utility, response, constant, configuration, and exception packages. The frontend follows a modular React structure with reusable components, page-level screens, route configuration, and Axios service files. This report explains the project background, objectives, architecture, database design, diagrams, module implementation, security flow, testing strategy, and future enhancements.

2. Introduction

Digital banking applications are expected to provide fast, secure, and user-friendly services. Users need a single platform to check account details, perform transactions, apply for loans, explore investments, identify risky transactions, and receive support. FinGenie AI addresses these requirements by combining banking operations and AI-oriented financial assistance into one application.

The system supports two major roles: Customer/User and Admin. The customer side focuses on personal banking operations such as dashboard access, account viewing, transactions, loan application, investment viewing, fraud risk checking, and chatbot support. The admin side focuses on monitoring and management activities such as viewing users, accounts, transactions, loans, investments, fraud alerts, and analytics. Role separation is important because normal users should only access their own data, while admins require a global view of the system.

3. Problem Statement

Traditional banking systems often keep services such as transaction history, fraud checking, investment guidance, loan prediction, and customer support as separate workflows. This increases user effort and makes the system less efficient. FinGenie AI solves this by integrating core banking operations, fraud risk analysis, loan prediction, investment advisory support, and chatbot assistance into a single platform.

The project also addresses the need for secure authentication and authorization. Since financial applications handle sensitive information, the system uses JWT-based authentication and Spring Security configuration. The frontend communicates with secured backend APIs and displays data according to the logged-in role. This ensures that customer and admin workflows remain logically separated and protected.

4. Objectives and Scope

The main objective of FinGenie AI is to develop a secure, modular, and user-friendly digital banking application with intelligent financial features. The project scope includes authentication, role-based dashboards, account management, transactions, fraud alerts, loans, investments, chatbot support, admin monitoring, backend API development, frontend integration, database mapping, and exception handling.

- To implement user and admin login flows with secure JWT authentication.
- To provide customer-specific modules where users can access only their own data.

- To provide admin modules where admins can view and manage complete application-level data.
- To implement banking operations such as deposit, withdrawal, transfer, and transaction history.
- To include AI-oriented features such as risk score generation, fraud alerting, loan eligibility prediction, credit score analysis, EMI calculation, investment suggestions, and chatbot assistance.
- To maintain clean separation between frontend UI, service integration, backend business logic, database access, and security configuration.

5. Technology Stack

The frontend is developed using ReactJS with JSX pages, reusable components, React Router for navigation, and Axios for API communication. The backend is developed using Java and Spring Boot with REST controllers, Spring Security, JWT, Spring Data JPA, and repository-based database access. Swagger/OpenAPI support is used for API documentation and testing, while Postman can be used for manual API validation. The project follows a layered architecture so that responsibilities are clearly separated and future changes can be handled easily.

Layer	Technologies / Files Used
Frontend	ReactJS, JSX, CSS, React Router DOM, Axios, reusable components, page modules and service files.
Backend	Java, Spring Boot, Spring Web, Spring Security, JWT, Spring Data JPA, DTOs, repositories and services.
Database Mapping	Entity classes such as User, Account, Transaction, Loan, Investment, FraudAlert, Admin, Role and ChatHistory.
Testing / Documentation	Swagger/OpenAPI and Postman for API testing and verification.

6. System Architecture

FinGenie AI follows a client-server architecture. The React frontend acts as the client layer and provides all user interfaces. The Spring Boot backend acts as the server layer and exposes REST APIs for authentication, banking operations, loans, investments, fraud alerts, and chat history. The database layer is accessed through repositories, and entity classes represent the database tables.

When a user or admin logs in, credentials are sent from the frontend to the authentication API. After successful validation, the backend returns a JWT token. This token is used in subsequent API requests. The JWT filter validates the token and allows access to secured endpoints only when the request is authenticated. Controllers receive API requests, services process business logic, repositories interact with database entities, and structured responses are sent back to the frontend.

7. Existing Project Structure

7.1 Backend Structure

The backend project is named FinGenieAI and is organized under the package com.fingenie. The main application class FinGenieAiApplication.java starts the Spring Boot application. SwaggerConfig.java is used for OpenAPI documentation. AppConstants.java stores common messages such as ACCOUNT_NOT_FOUND, TRANSACTION_NOT_FOUND, and USER_NOT_FOUND so repeated strings are avoided in the codebase.

The controller package contains REST controllers such as AccountController, AdminController, AdminAuthController, AuthController, ChatHistoryController, FraudAlertController, InvestmentController, LoanController, TransactionController, and UserController. These controllers expose endpoints and pass requests to the service layer. DTO classes such as AccountDTO, AdminDTO, AdminLoginDTO, AuthResponseDTO, ChatHistoryDTO, FraudAlertDTO, InvestmentDTO, LoanDTO, LoginDTO, RegisterDTO, TransactionDTO, and UserDTO transfer data safely between frontend and backend.

The entity package contains the core domain classes: User, Admin, Account, Transaction, Loan, Investment, FraudAlert, Role, and ChatHistory. Repositories perform database operations. Service interfaces define required operations, and service implementation classes contain business logic. The security package contains CustomUserDetailsService, JwtAuthenticationEntryPoint, JwtFilter, JwtUtil, and SecurityConfig. The util package includes LoanPredictionEngine, and the exception package contains custom exceptions and global exception handling classes.

7.2 Frontend Structure

The frontend project is named FINGENIE-FRONTEND. The src/components folder contains reusable components such as AdminBackButton.jsx, BackButton.jsx, Footer.jsx, Navbar.jsx, and Sidebar.jsx. The src/pages folder contains user pages such as Login.jsx, Register.jsx, Dashboard.jsx, Accounts.jsx, Transactions.jsx, Loan.jsx, Investments.jsx, FraudAlerts.jsx, and ChatAssistant.jsx. Admin pages include AdminLogin.jsx, AdminDashboard.jsx, AdminAccounts.jsx, AdminTransactions.jsx, AdminLoans.jsx, AdminInvestments.jsx, and AdminFraudAlerts.jsx.

The src/routes folder contains AppRoutes.jsx, which defines frontend navigation paths and separates user/admin routing. The src/services folder contains api.js and module-wise service files such as authService.js, accountService.js, transactionService.js, loanService.js, investmentService.js, fraudService.js, and chatService.js. These service files keep backend integration separate from UI code and make the frontend easier to maintain.

8. Design Diagrams and Explanation

This section presents the five main design diagrams used to describe the structure and behavior of the FinGenie AI application. These diagrams explain the database design, data flow, user interactions, class relationships, and runtime sequence of important modules.

8.1 ER Diagram

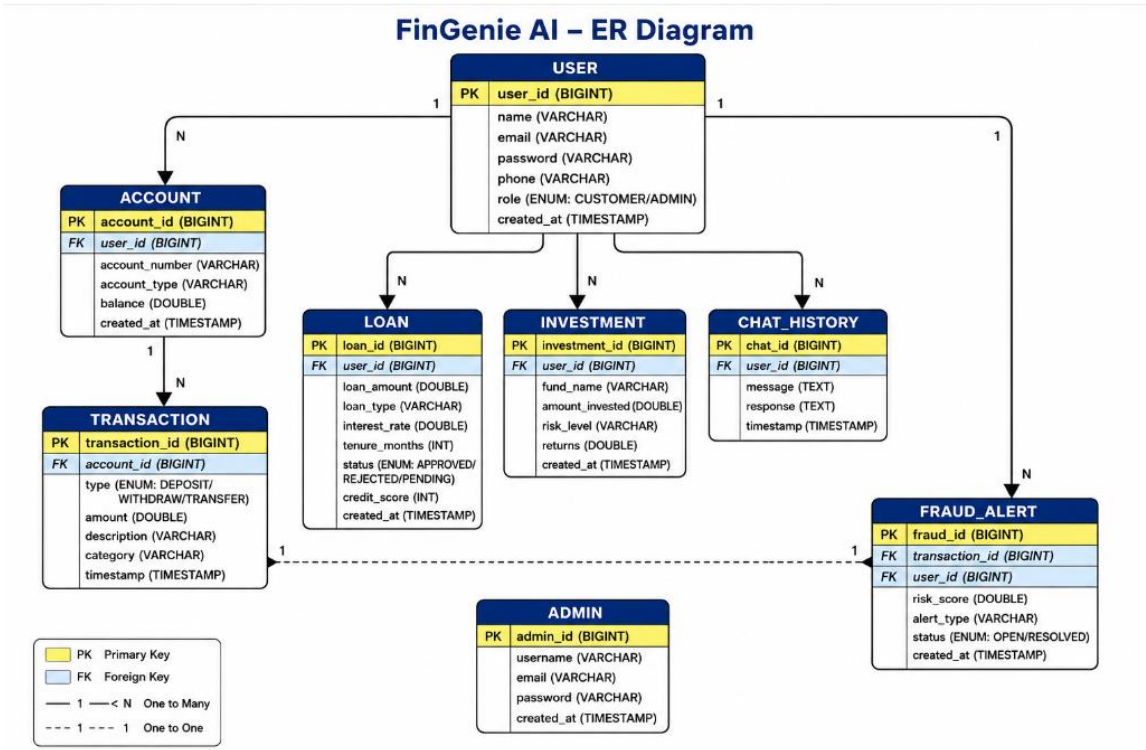


Figure: 8.1 ER Diagram

The ER diagram represents the database design of the FinGenie AI banking system. The User table is the central table and stores user_id, name, email, password, phone, role, and created_at. A user can have multiple accounts, loans, investments, chat history records, and fraud alerts. This supports the requirement that each logged-in customer should see only their own data.

The Account table is linked to the User table using user_id as a foreign key. Each account can contain many transactions. The Transaction table stores transaction_id, account_id, type, amount, description, category, and timestamp. FraudAlert is connected to both Transaction and User, allowing the system to identify which user and which transaction caused a fraud warning. Loan and Investment store financial planning data linked to the user, while Admin is represented separately for admin authentication and monitoring activities.

8.2 DFD Diagram

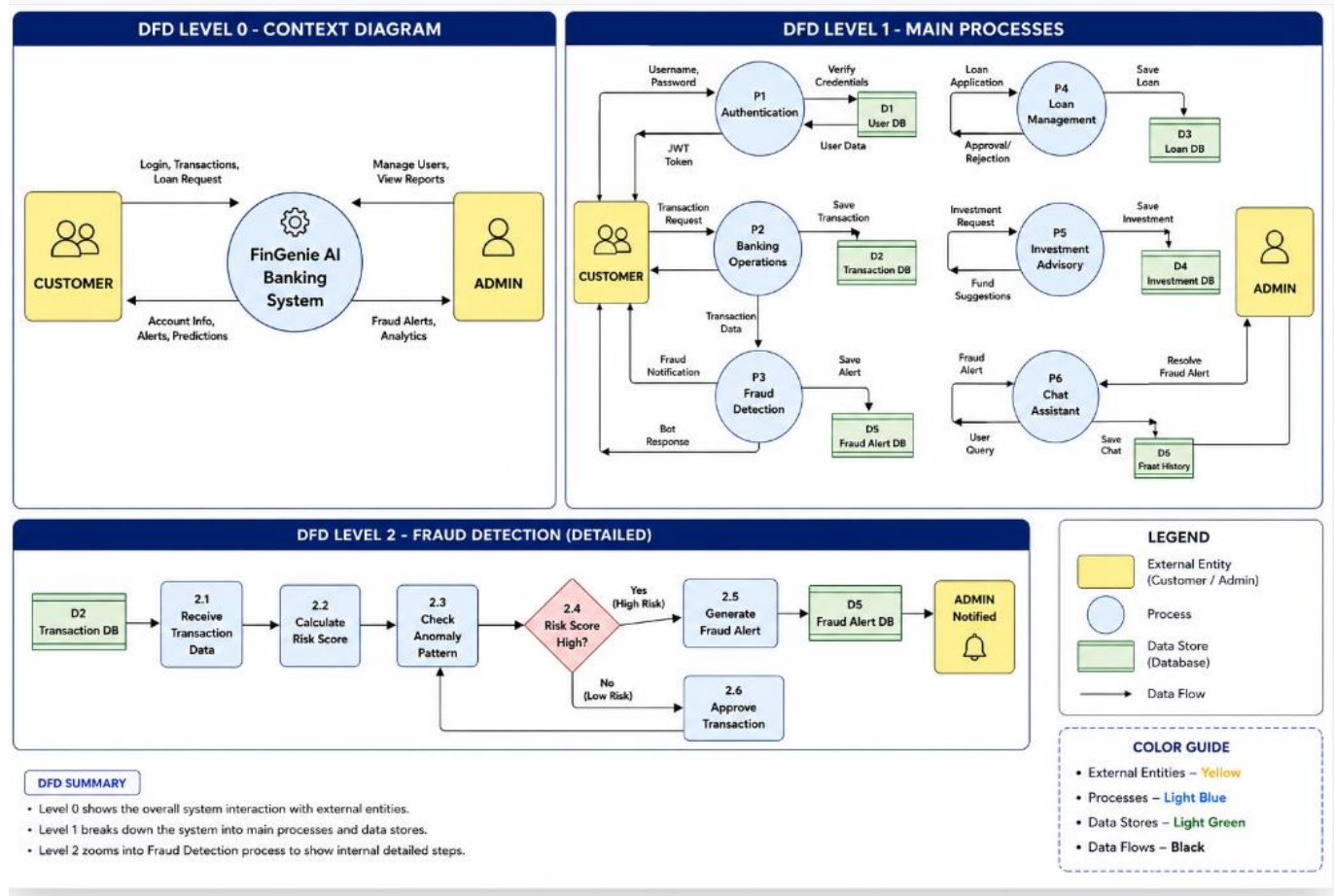


Figure: 8.2 DFD Diagram

The DFD diagram explains how data moves through the system. At Level 0, FinGenie AI is shown as one banking system interacting with two external entities: Customer and Admin. The customer sends login requests, transactions, and loan requests and receives account information, alerts, and predictions. The admin manages users, views reports, receives fraud alerts, and analyzes system-level information.

At Level 1, the system is divided into Authentication, Banking Operations, Fraud Detection, Loan Management, Investment Advisory, and Chat Assistant processes. These processes communicate with data stores such as User DB, Transaction DB, Loan DB, Investment DB, Fraud Alert DB, and Chat History. Level 2 expands the fraud detection process: transaction data is received, risk score is calculated, anomaly pattern is checked, and high-risk transactions generate fraud alerts while low-risk transactions are approved.

8.3 Use Case Diagram

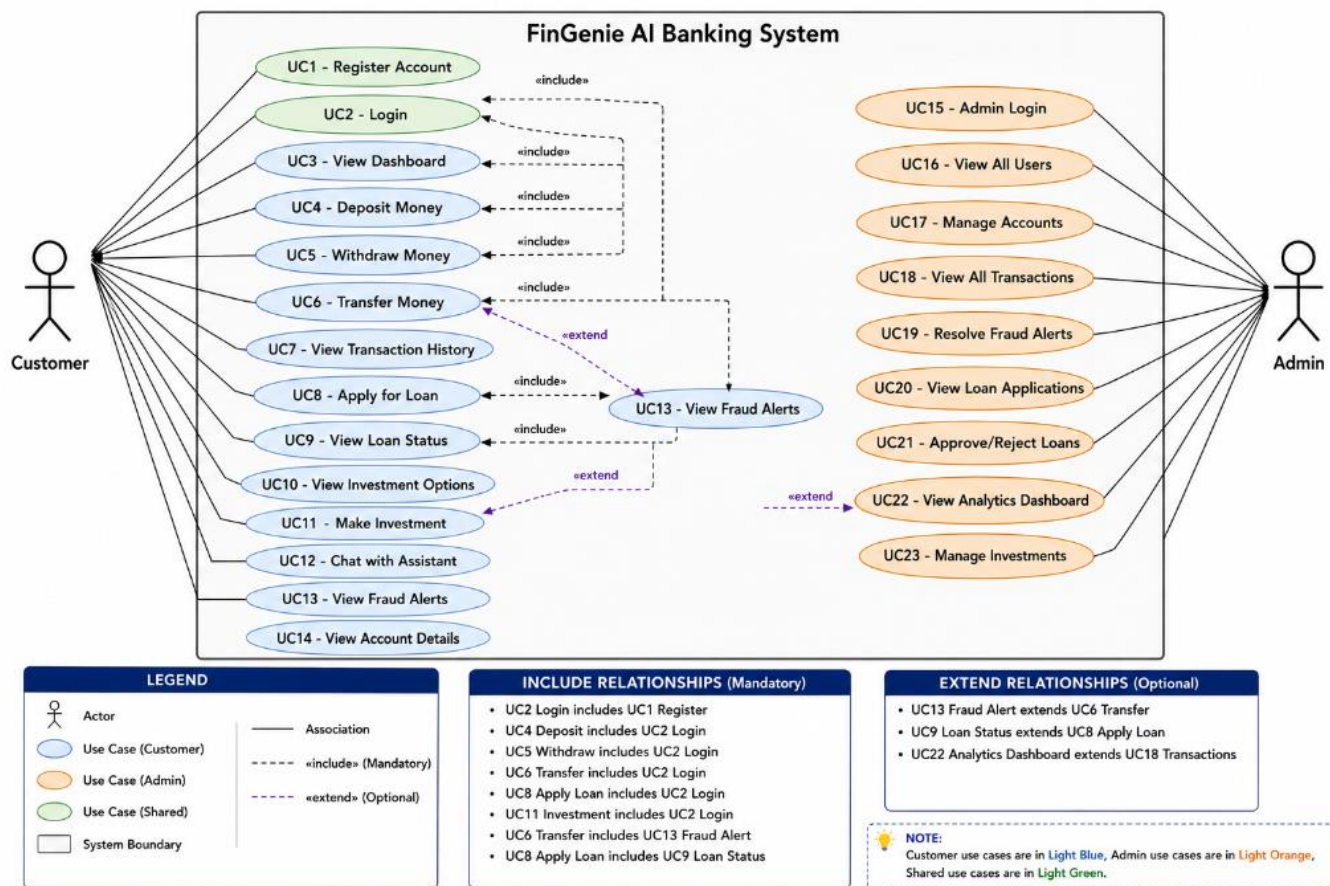


Figure: 8.3 Use Case Diagram

The use case diagram shows how Customer and Admin actors interact with the FinGenie AI Banking System. The customer can register an account, login, view dashboard, deposit money, withdraw money, transfer money, view transaction history, apply for loan, view loan status, view investment options, make investments, chat with assistant, view fraud alerts, and view account details.

The admin actor can perform admin login, view all users, manage accounts, view all transactions, resolve fraud alerts, view loan applications, approve or reject loans, view analytics dashboard, and manage investments. Include relationships show mandatory dependencies such as login before banking operations, while extend relationships show optional flows such as fraud alert generation during transfer and analytics extension from transaction viewing.

8.4 Class Diagram

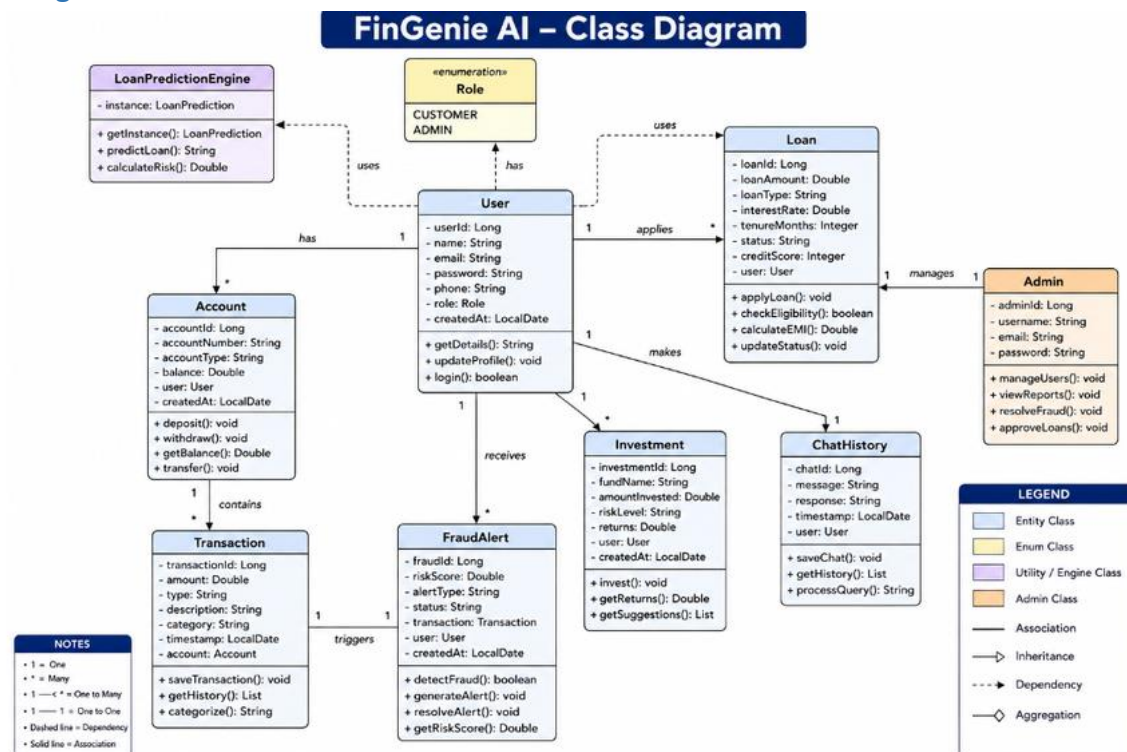


Figure: 8.4 Class Diagram

The class diagram represents the object-oriented structure of the system. User stores profile and authentication-related details and is associated with Account, Loan, Investment, FraudAlert, and ChatHistory. The Role enumeration contains CUSTOMER and ADMIN values, which support role-based behavior in the application.

Account contains account number, account type, balance, user reference, and created date, and supports deposit, withdraw, getBalance, and transfer operations. Transaction stores amount, type, description, category, timestamp, and account reference. FraudAlert stores risk score, alert type, status, transaction reference, user reference, and created date, and supports detecting fraud, generating alerts, resolving alerts, and calculating risk score.

Loan stores loan amount, loan type, interest rate, tenure, status, credit score, and user reference. It supports applying for loan, checking eligibility, calculating EMI, and updating loan status. Investment stores fund name, amount invested, risk level, returns, and user reference. ChatHistory stores chatbot messages and responses. Admin manages users, views reports, resolves fraud cases, and approves loans. LoanPredictionEngine is a utility/engine class that provides prediction and risk calculation support.

8.5 Sequence Diagram

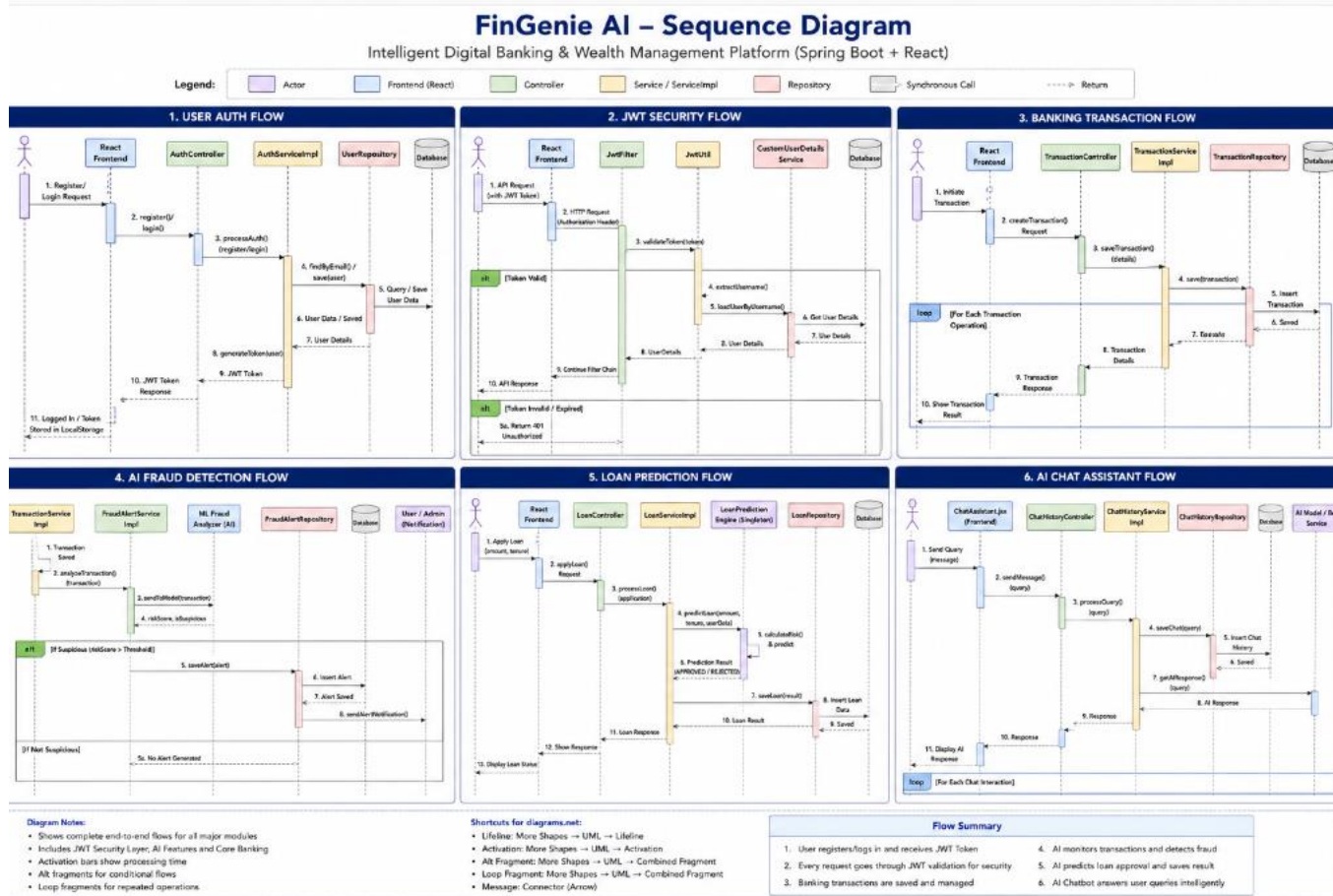


Figure: 8.5 Sequence Diagram

The sequence diagram describes the runtime flow of important modules. The user authentication flow starts from the React frontend and reaches AuthController, AuthServiceImpl, UserRepository, and Database. After successful registration or login, the backend generates a JWT token and returns it to the frontend. The frontend stores the token and uses it for future secured requests.

The JWT security flow shows how an API request with Authorization header passes through JwtFilter and JwtUtil. If the token is valid, user details are loaded using CustomUserDetailsService and the request continues to the controller. If the token is invalid or expired, an unauthorized response is returned. Banking transaction flow shows how the frontend calls TransactionController, TransactionServiceImpl, TransactionRepository, and Database.

The AI fraud detection flow shows how transaction data is passed to FraudAlertServiceImpl and a risk checker. If the calculated risk is above the threshold, a fraud alert is saved and a notification can be generated. The loan prediction flow shows LoanController, LoanServiceImpl, LoanPredictionEngine, LoanRepository, and Database. The chatbot flow shows ChatAssistant.jsx sending messages to ChatHistoryController and receiving a response from a chat/AI response service.

9. Functional Modules

FinGenie AI contains several functional modules: authentication and role management, account and banking operations, fraud detection, loan management, investment advisor, and AI chat banking assistant. Each module has a frontend page, service

integration file, backend controller, service layer, repository, DTO, and entity support wherever required. This modular design improves readability and allows each feature to be enhanced independently.

10. API Integration Layer

The frontend service layer keeps API logic separate from page UI logic. `api.js` generally contains the Axios instance and backend base URL configuration. Other service files call specific backend endpoints. For example, `authService.js` handles login and registration calls, `accountService.js` handles account APIs, `transactionService.js` handles transaction APIs, `loanService.js` handles loan APIs, `investmentService.js` handles investment APIs, `fraudService.js` handles fraud alert APIs, and `chatService.js` handles chatbot-related APIs.

11. Security Implementation

Security is implemented using Spring Security and JWT. `SecurityConfig` defines public and secured endpoints. `JwtUtil` generates and validates tokens. `JwtFilter` intercepts incoming requests and checks whether the Authorization header contains a valid token. `CustomUserDetailsService` loads user details required by Spring Security, and `JwtAuthenticationEntryPoint` handles unauthorized access attempts. This protects banking and admin APIs from unauthenticated access.

12. Database Design Explanation

The database design is based on entities such as User, Admin, Account, Transaction, Loan, Investment, FraudAlert, Role, and ChatHistory. User is the central entity for customer data. Account is linked to User, and Transaction is linked to Account. Loan, Investment, FraudAlert, and ChatHistory are also connected to User. This relationship design supports user-specific filtering because records can be retrieved based on the logged-in user identifier.

13. Exception Handling and Validation

The backend includes a dedicated exception package containing `DuplicateRecordException`, `ErrorResponse`, `GlobalExceptionHandler`, `LoanNotEligibleException`, `ResourceNotFoundException`, and `UnauthorizedException`. `GlobalExceptionHandler` centralizes error handling so controllers do not need repeated try-catch logic. `ApiResponse<T>` standardizes success responses with success, message, and data fields.

14. User and Admin Workflows

The user workflow begins with registration or login and then continues to `Dashboard.jsx`. From the dashboard, users can view account details, perform transactions, check transaction history, use risk checker, apply for loans, calculate EMI, analyze credit score, view investments, check fraud alerts, and use the chat assistant. The admin workflow begins with `AdminLogin.jsx` and continues to `AdminDashboard.jsx`, where admins can view and manage accounts, transactions, loans, investments, fraud alerts, users, and analytics.

15. Testing Strategy

Testing covers authentication, frontend navigation, backend APIs, role separation, module behavior, and exception handling. Authentication testing verifies valid login, invalid login, token generation, and unauthorized access. Role testing verifies that customers see only their own data while admins see complete platform data. API testing can be performed using Swagger UI or Postman, and frontend testing verifies forms, buttons, navigation, dashboard links, response display, and error message handling.

16. Rubrics Coverage

Evaluation Area	Project Coverage
React UI & Dashboard Design	User and admin dashboards are implemented with separate pages and reusable components.
React State Management	React pages manage form values, API responses, login flow, dashboard navigation, and module data.
Spring Boot REST APIs	Controllers expose REST APIs and services handle business logic for major modules.
Banking Transaction Logic	Transaction module supports banking operations and transaction history.
AI Fraud Detection Logic	Fraud alert and transaction risk score features support suspicious transaction analysis.
AI Loan Prediction	Loan module and LoanPredictionEngine support loan eligibility and prediction workflow.
JWT Security	SecurityConfig, JwtUtil, JwtFilter, and CustomUserDetailsService support secured API access.
Database Design	Entity and repository layers cover user, account, transaction, loan, investment, fraud, and chat data.
Frontend-Backend Integration	Axios service files connect React pages with Spring Boot APIs.
Exception Handling & Validation	Custom exceptions and GlobalExceptionHandler provide structured error handling.

17. Execution Steps

To run the backend, open the FinGenieAI project in STS or Eclipse, configure database properties in application.properties, and run FinGenieAiApplication.java as a Spring Boot application. Swagger UI or Postman can be used to verify APIs. JWT-protected endpoints should be tested with a valid Authorization token.

To run the frontend, open FINGENIE-FRONTEND in VS Code, run npm install if dependencies are not already installed, verify that api.js points to the backend base URL, and run npm start. After the frontend starts, user login and admin login can be tested separately to verify role-based navigation and API integration.

18. Future Enhancements

- Integrate real email/SMS OTP for production-level MFA instead of demo-only verification.
- Replace rule-based fraud and loan logic with trained machine learning models for more accurate predictions.
- Add charts for portfolio analytics, transaction trends, loan statistics, and admin dashboard insights.
- Add audit logging for sensitive banking operations and admin actions.
- Add notification services for fraud alerts, loan status updates, and investment reminders.
- Deploy frontend and backend to cloud platforms with CI/CD pipeline and production database configuration.

19. Conclusion

FinGenie AI is a complete Java full-stack case study project that combines digital banking operations, role-based authentication, fraud risk analysis, loan prediction, investment support, chatbot assistance, and admin monitoring. The application follows a clean layered backend architecture and a modular React frontend structure. The use of DTOs, services, repositories, security classes, exception handling, and reusable frontend components improves maintainability and scalability.

The included ER, DFD, Use Case, Class, and Sequence diagrams explain the database structure, system flow, actor interactions, object-oriented design, and runtime behavior of the application. Overall, the project satisfies the major case study expectations such as React UI design, Spring Boot REST APIs, transaction logic, JWT security, database design, AI-oriented modules, frontend-backend integration, exception handling, and documentation.